

Tópico 2 – DSW (Modelagem)

Módulos

Módulo Projeto Orientado a Abordagens Tradicionais (Notação UML - Diagramas Estáticos)

Módulo Projeto Orientado a Abordagens Tradicionais (Notação UML - Diagramas Dinâmicos)



Módulo Projeto Orientado a Abordagens Tradicionais

Conteúdos

Diagramas Estáticos

Artefatos Relevantes:

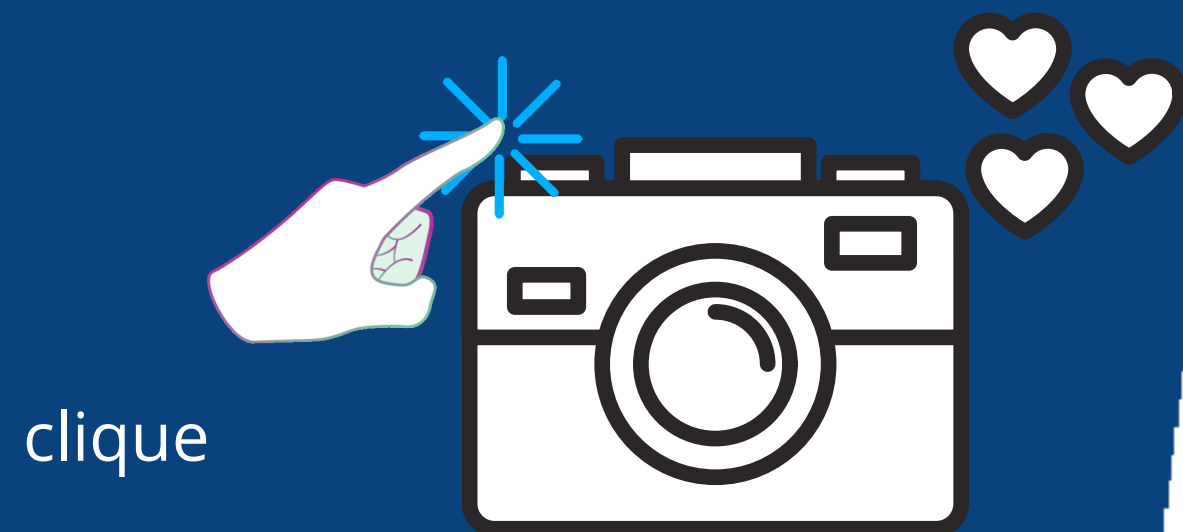
- Diagrama de Classes
- Diagrama de Pacotes
- Diagrama de Componentes

Há outros diagramas estáticos relevantes.
Entretanto, ficaremos nesses principalmente...



Diagramas Estáticos

Foco: Como o próprio nome sugere, tem-se uma visão mais estática do sistema, como se estivéssemos tirando um foto do nosso sistema (ou de parte dele), em um dado momento.



- matricula
- nome
- email
- periodo
- ...

Classe Estudante



- matricula
- nome
- email
- especializacao
- ...

Classe Professor



0..*
cursa
0..*

1..*
ministra

0..*

- codigo
- nome
- ementa
- curso
- ...

Classe Disciplina



Padronização & UML

Para garantir que outros possam compreender de forma adequada nosso modelo estático, precisamos modelar orientados a uma notação conhecida, respeitada pela comunidade.

Hoje, temos muitas notações respeitadas e reconhecidas na comunidade, para diferentes propostas de modelagem.



- Linguagem de Modelagem Unificada
- Padronizada
- Reconhecida
- Respeitada
- Muitos Níveis de Modelagem Oferecidos
- Vasto Arcabouço de Ferramentas

Notação UML - Principais Diagramas Estáticos



- **Diagrama de Classes**
 - **Foco, de cunho estático, nas classes, nos atributos, nos relacionamentos, nas cardinalidades, nas semânticas, no escopo/na visibilidade.** Repare que esse modelo é estático, técnico, próximo ao código, e bem relevante no desenvolvimento de um projeto de software. Além disso, esse modelo contém muitas ricas informações sobre o software. Se modelado de forma equivocada, pode induzir a erros de navegabilidade, visibilidade, propriedades, métodos, e assim vai...
- **Diagrama de Pacotes**
 - **Foco, de cunho estático, na organização dos principais pacotes.** Aqui, reparem que essa organização dos pacotes já remete à organização do código em si. Portanto, esse modelo é estático, organizacional, técnico, e bem "espelho" do código. Além disso, esse nível de modelagem já revela detalhes em termos arquiteturais.
- **Diagrama de Componentes**
 - **Foco, de cunho estático, no intuito de modularizar o sistema, pensando mesmo em componentes reutilizáveis, desenvolvimento orientado a equipes distribuídas, com diferentes competências e habilidades.** Reparem que os componentes são unidades coesas, que podem ser desenvolvidas por equipes trabalhando de forma independente, pelo menos em parte. Aqui, alta coesão e baixo acoplamento são princípios chave!

Diagrama de Classes

Para tornar o conhecimento sobre esse nível de modelagem mais concreto, vamos pensar em um software que pretendemos desenvolver...

Imaginando ser um PetShop!

Temos, em um primeiro desenho da Baseline de Requisitos, que o software será focado no cadastro de clientes, pets, e serviços prestados. Além disso, será um site, permitindo apenas o acompanhamento do serviço, com os seguintes status apresentados ao cliente:

- Serviço contratado
- Iniciando o serviço
- Serviço em andamento
- Finalizando o serviço
- Serviço realizado

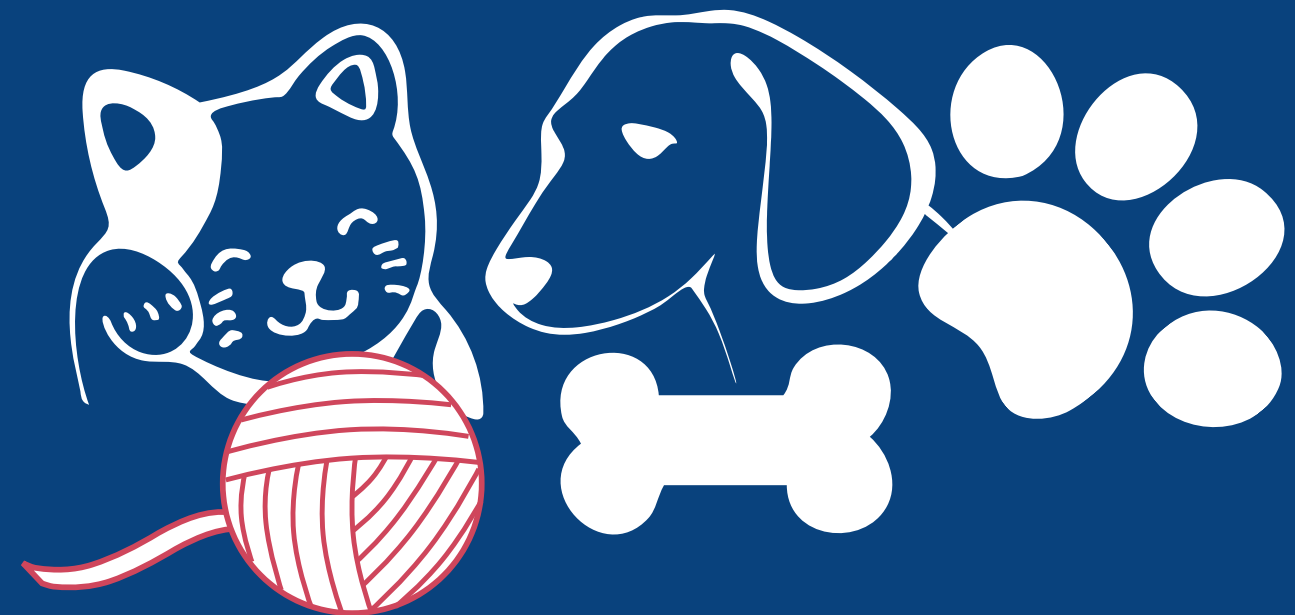


Diagrama de Classes



Se a nossa intenção é justamente modelar o Diagrama de Classes, precisaremos focar nas entidades, ou seja, nas classes. Essas classes são formadas por conjuntos de objetos, que possuem propriedades e métodos.

Propriedades e métodos possuem escopo/visibilidade. Propriedades podem ser constantes ou variáveis. Propriedades possuem tipos, se forem variáveis. Valores precisam constar especificados, se forem constantes.

Além disso, precisaremos perceber como essas classes se relacionam entre si, ou seja, precisaremos pensar nos relacionamentos. Se temos relacionamentos, teremos semânticas e cardinalidades associadas, além da necessidade de especificar a navegabilidade.

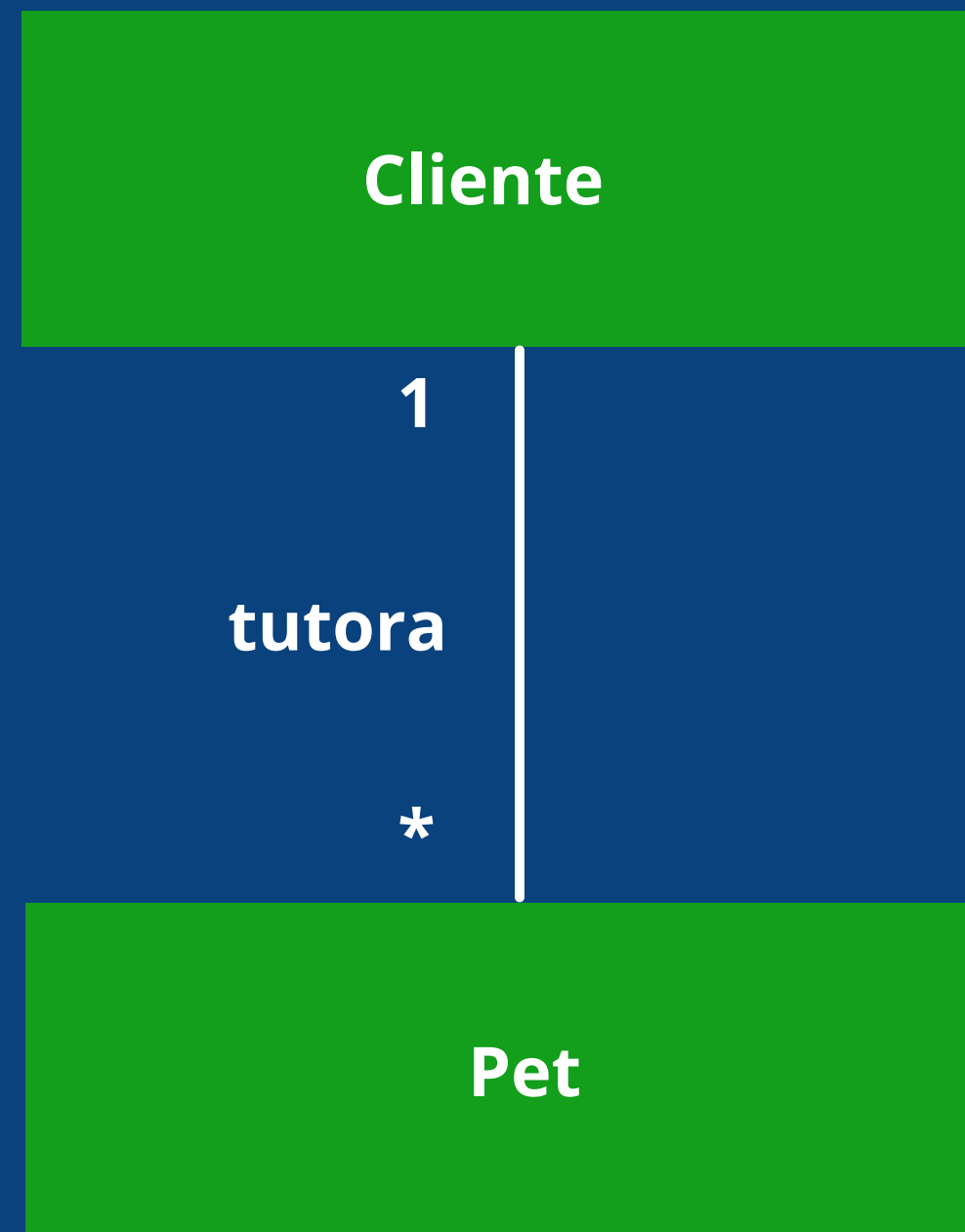
E assim vai...

Portanto, percebam que, para se ter um modelo de classes bem especificado, muitos aspectos precisam ser considerados.

Para cada detalhe da especificação, tem-se que a UML padroniza de alguma forma.

Caso contrário, seria muito complicado compreender modelos desenhados, cada qual, de forma diferente.

Diagrama de Classes - Primeiro Esboço



Em um primeiro momento, vamos especificar as classes **Cliente** e **Pet**.

Reparem:

- Nomes de Classes no SINGULAR, e com as Iniciais em MAIÚCULO.
- tem-se a semântica no relacionamento, no caso: "tutora"
- têm-se as cardinalidades especificadas.

Diagrama de Classes - Primeiro Esboço

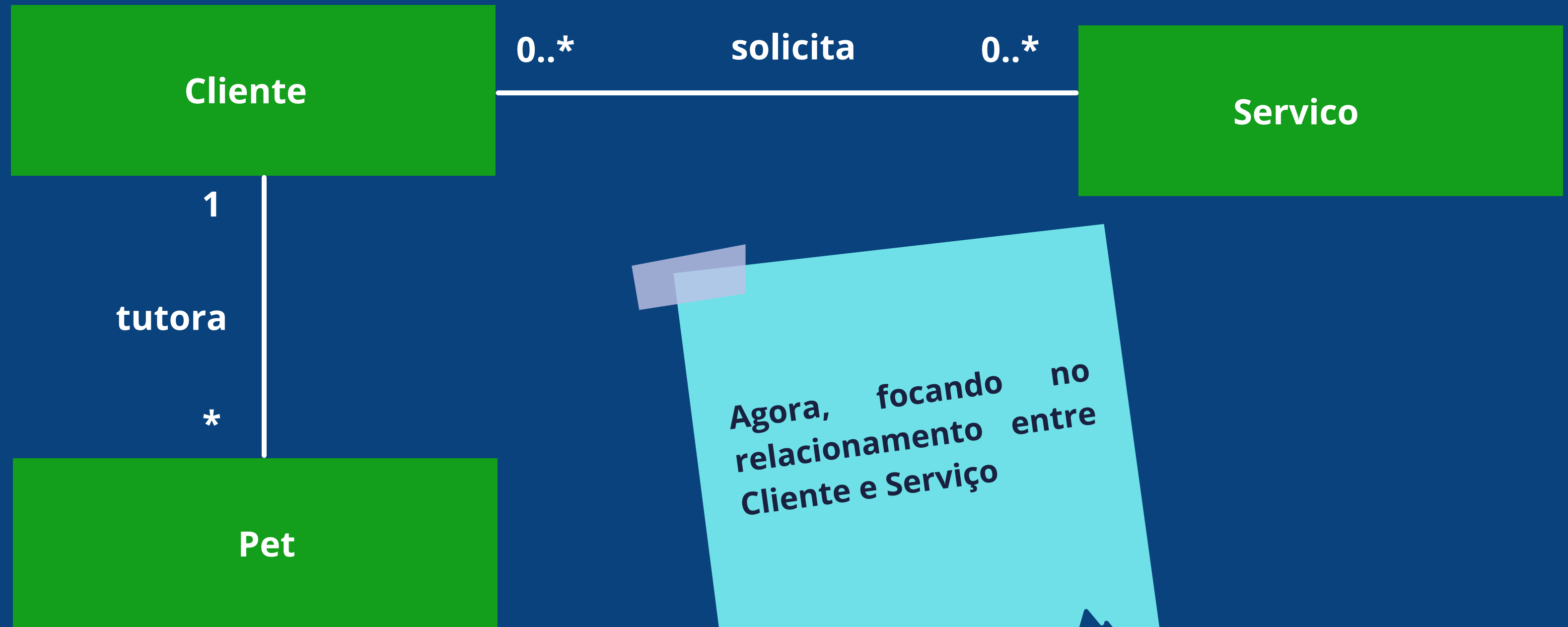
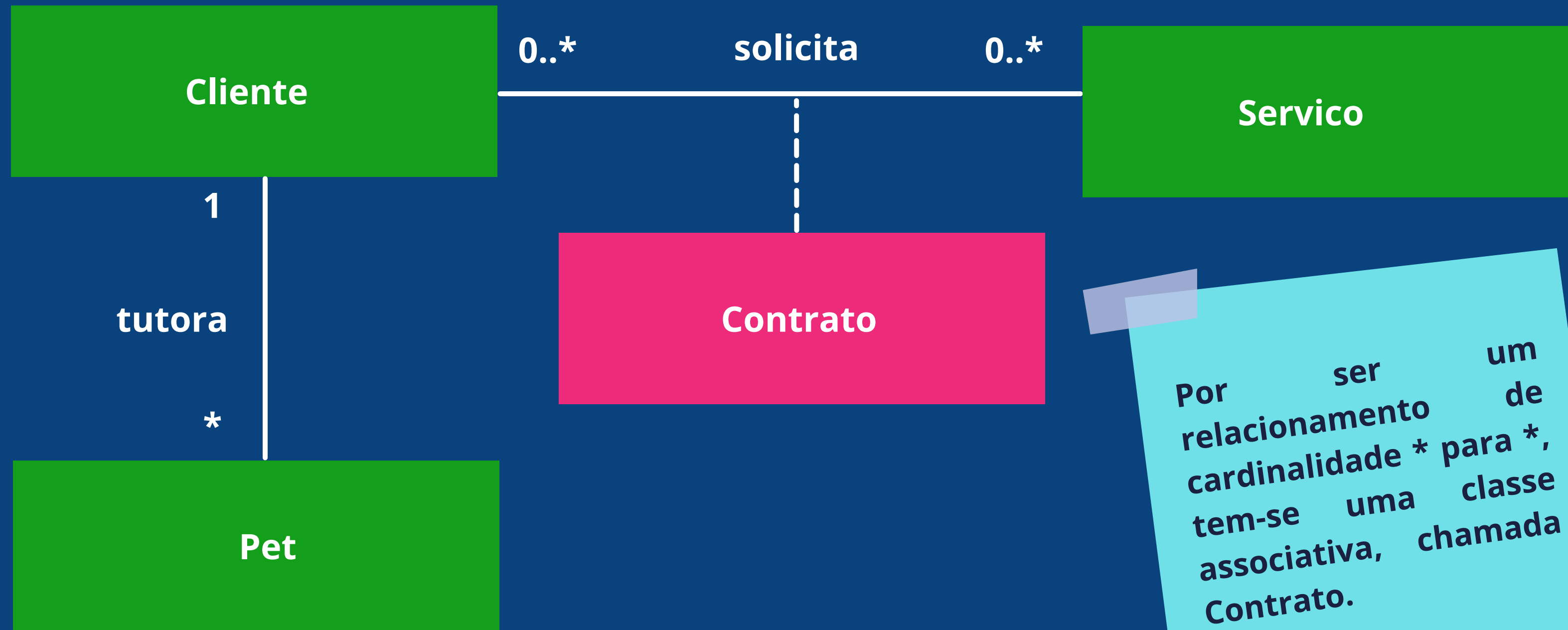
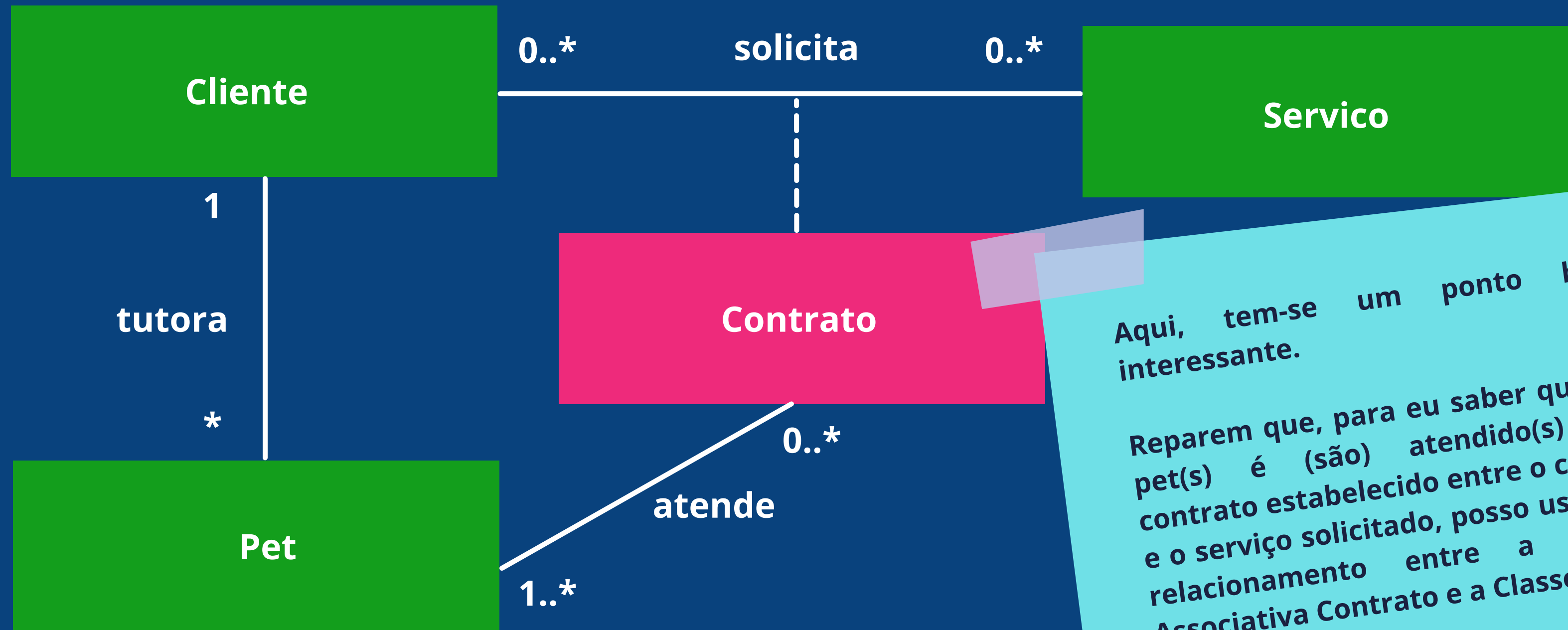


Diagrama de Classes - Primeiro Esboço



Por ser um relacionamento de cardinalidade * para *, tem-se uma classe associativa, chamada Contrato.

Diagrama de Classes - Primeiro Esboço

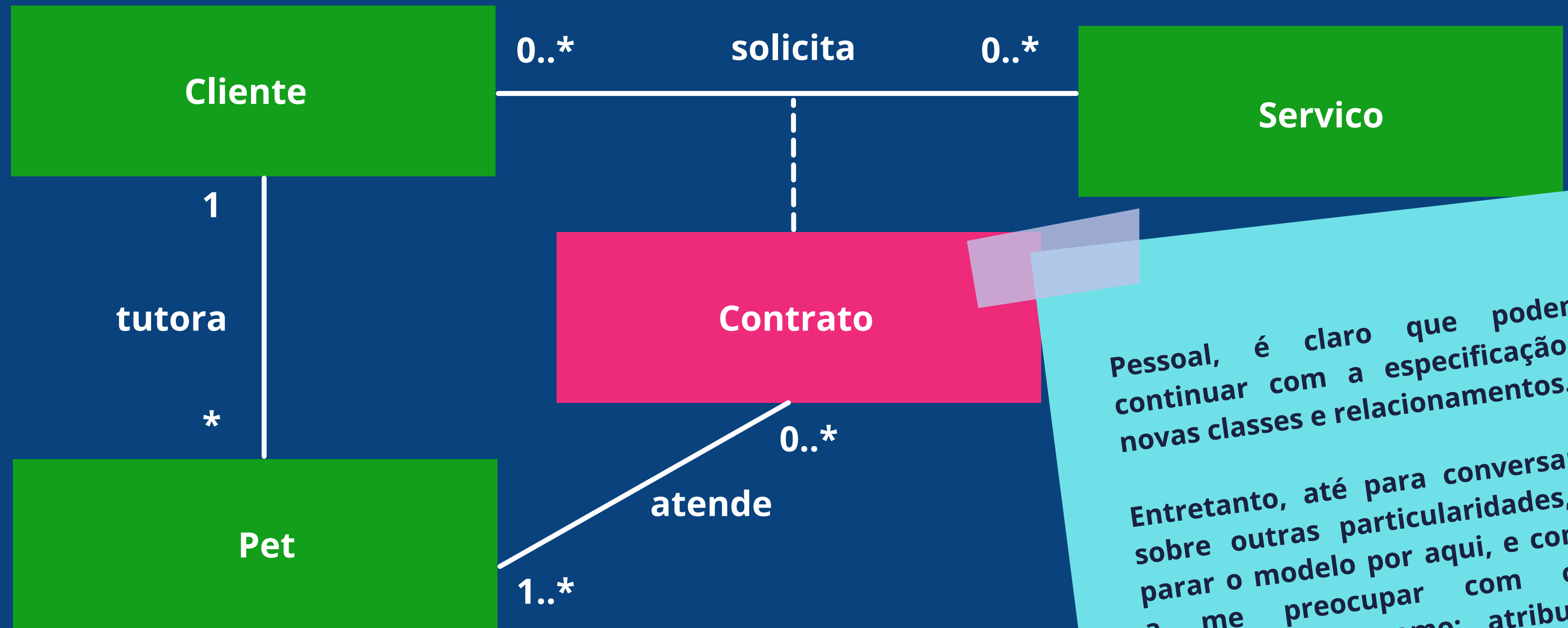


Aqui, tem-se um ponto bem interessante.

Reparem que, para eu saber qual(is) pet(s) é (são) atendido(s) via contrato estabelecido entre o cliente e o serviço solicitado, posso usar um relacionamento entre a Classe Associativa Contrato e a Classe Pet.

Poderia usar um ternário também...

Diagrama de Classes - Primeiro Esboço



Pessoal, é claro que podemos continuar com a especificação de novas classes e relacionamentos.

Entretanto, até para conversarmos sobre outras particularidades, vou parar o modelo por aqui, e começar a me preocupar com outros aspectos, tais como: atributos e métodos.

Diagrama de Classes - Atributos

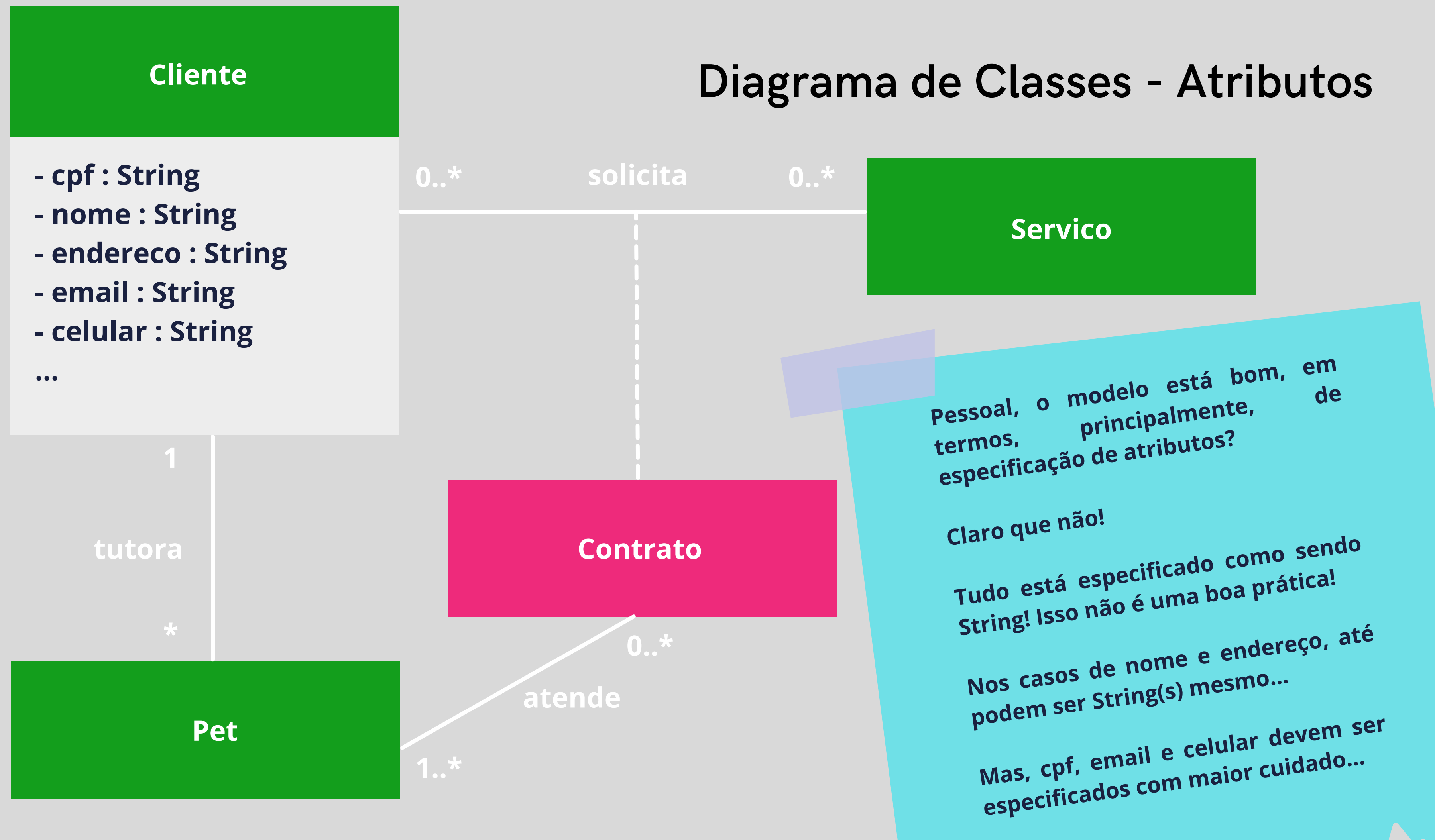


Diagrama de Classes - Atributos

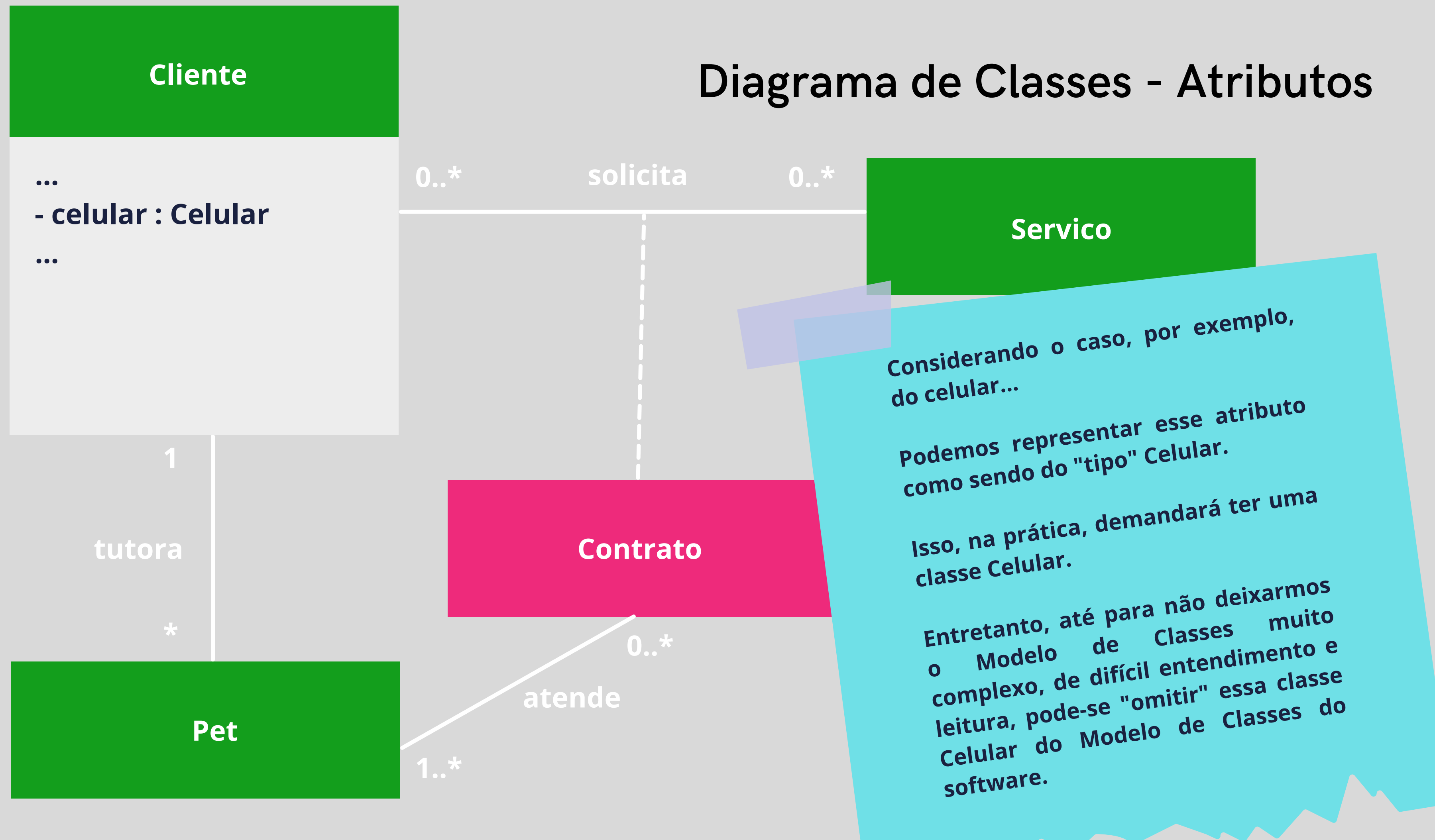


Diagrama de Classes - Atributos

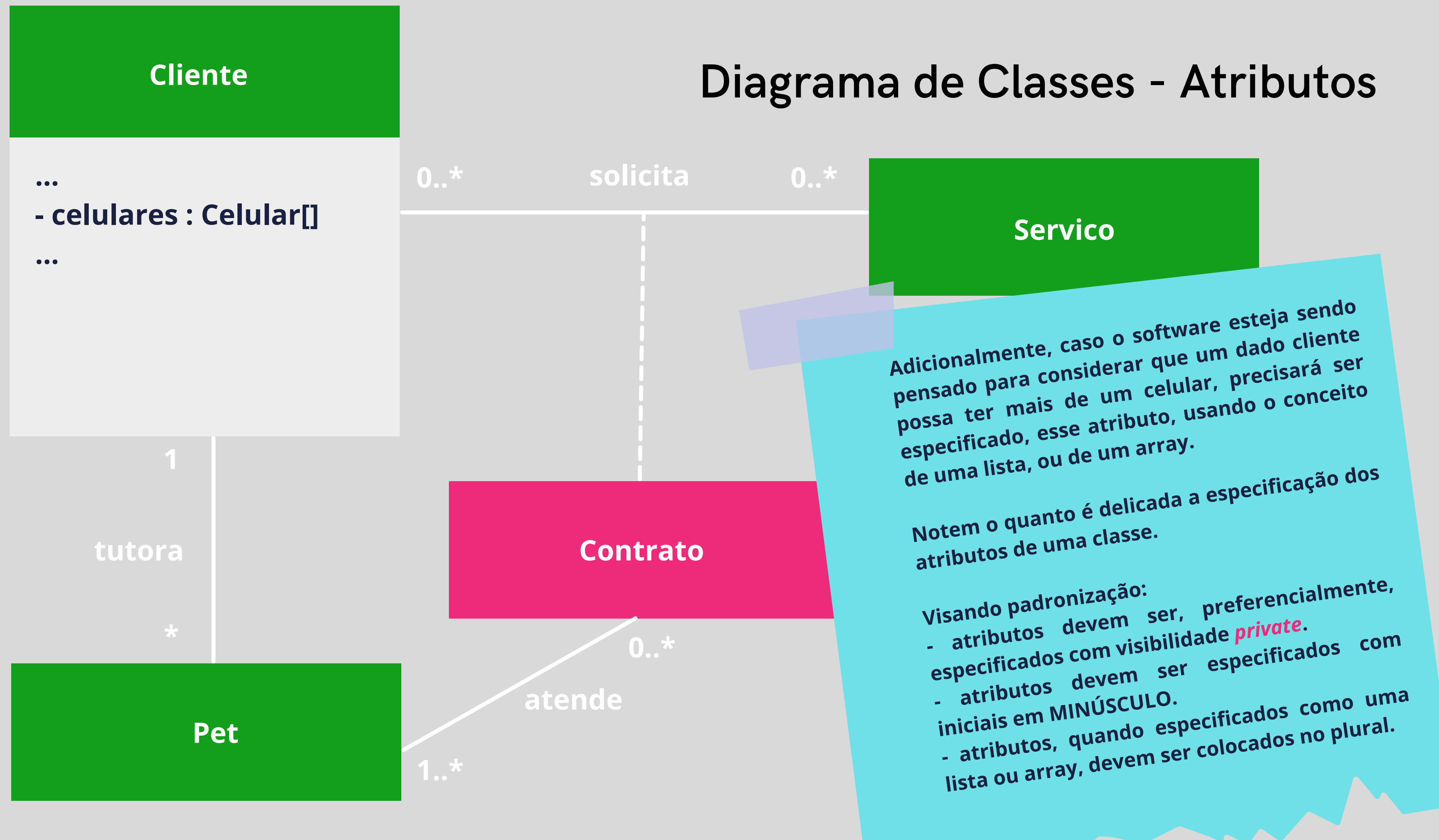
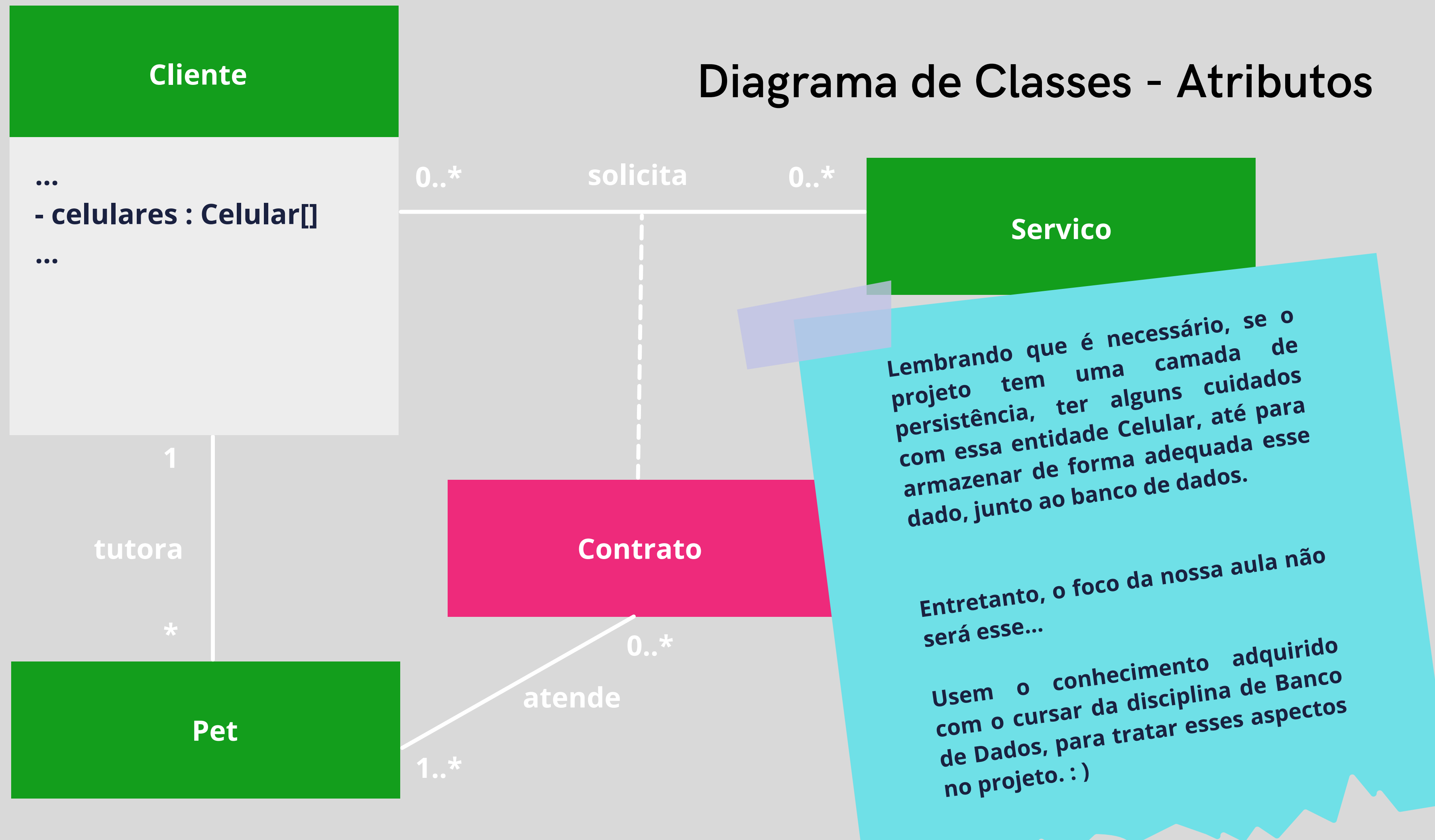


Diagrama de Classes - Atributos



Cuidado 1

Pensem, se é necessário especificar um valor de entrada, para um dado atributo. Isso pode evitar problemas na programação.

```
ehPrioritario : boolean[1] = false
```

Cuidado 2

Pensem, se há necessidade de especificar constantes.

Afinal, focamos nos atributos variáveis, até o momento..

TEMPO_MAX_MINUTOS : Integer = 120 {readOnly} //mais completo

frozen

- Podem ocorrer outras variações ainda, dependendo da ferramenta utilizada...
- Entretanto, tenham em mente que a especificação de CONSTANTES é com TUDO EM MAIÚSCULO.

Cuidado 3

Pensem, se um dado atributo possui restrições.

```
clientes : Cliente[*] {ordenados}
```

Cuidado 4

Pensem, se é necessário especificar um atributo estático.

Mas, o que é?

Um atributo estático é aquele, cujo valor vale para todos os objetos de uma dada classe, independentemente da instância.

nroDeAtendimentosPorFuncionario: Integer = 3 //sublinhado

- Pode variar, dependendo da ferramenta utilizada.

Cuidado 5

Lembrem-se sempre que uma coisa é a representação de um atributo no nível de modelagem, que é mais conceitual, generalista, e não atrelada à uma linguagem de programação específica. Portanto, trata-se de uma representação mais abstrata.

Já a representação de um atributo no código é mais concreta, e deve atender as particularidades da linguagem de programação.

Exemplo:

public static final PI = 3.1415 -> PI : Double = 3.1415 {readOnly}

Cuidado 6

Pensem, se há atributos derivados.
/dataFim : Date = dataInicio + duracao

Cuidado 7

Pensem sobre multiplicidades, no nível do atributo.
Isso ocorre, por exemplo, para arrays e matrizes.

patrocinadores : Patrocinador[10] {deve ter pelo menos 1 patrocinador}



multiplicidade

Tenham em mente....

[visibilidade] [/] nome [:tipo] [multiplicidade] [=default] [{propriedades}]

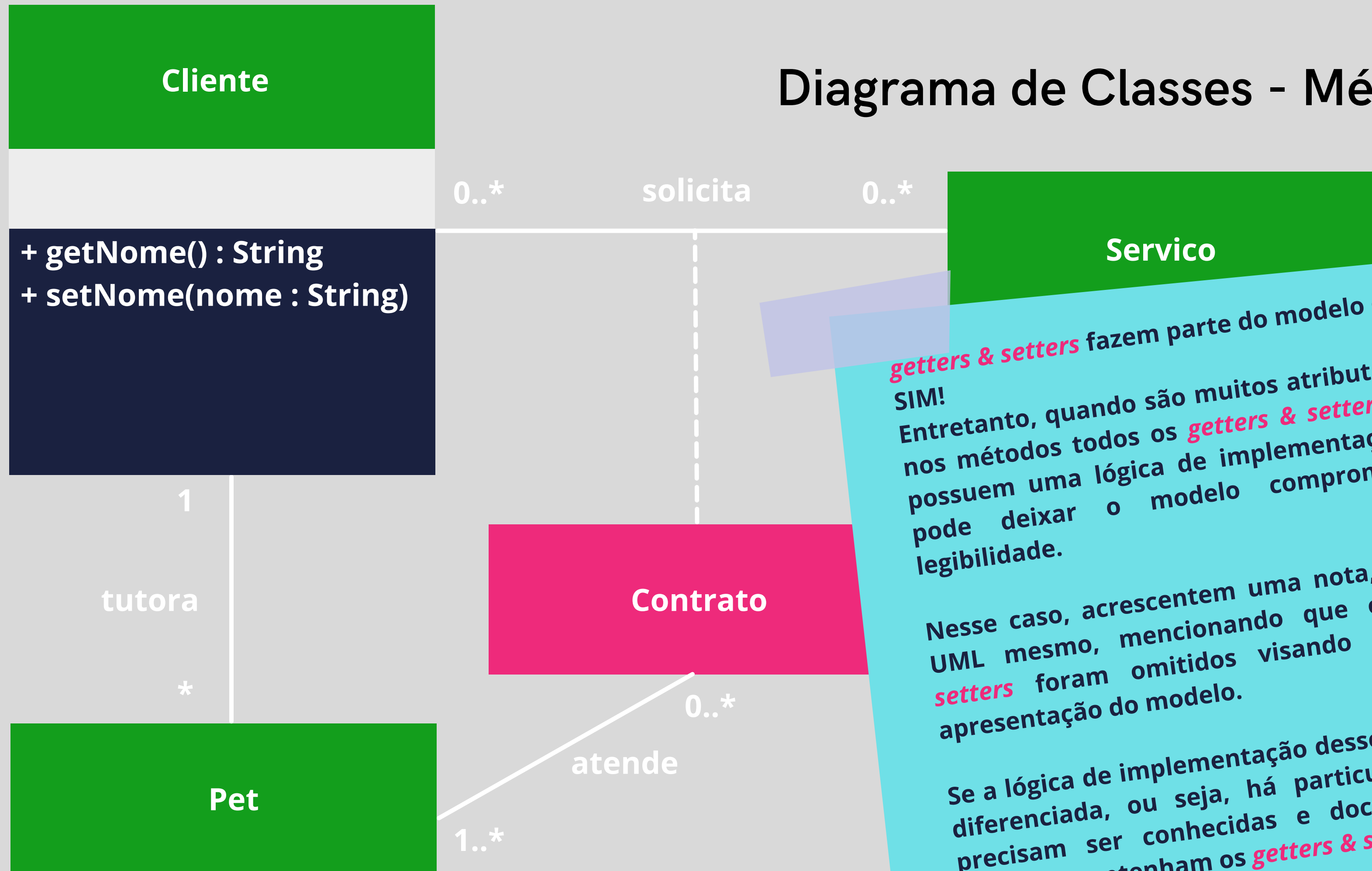


o símbolo / define um atributo derivado

Boa parte é opcional!

Entretanto, quanto mais completo, melhor...

Diagrama de Classes - Métodos



getters & setters fazem parte do modelo de classes SIM!
Entretanto, quando são muitos atributos, colocar nos métodos todos os **getters & setters**, se esses possuem uma lógica de implementação default, pode deixar o modelo comprometido em legibilidade.

Nesse caso, acrescentem uma nota, na notação UML mesmo, mencionando que os **getters & setters** foram omitidos visando uma melhor apresentação do modelo.

Se a lógica de implementação desses métodos for diferenciada, ou seja, há particularidades que precisam ser conhecidas e documentadas no modelo, mantenham os **getters & setters**.

Tenham em mente....

[visibilidade] nome (lista-de-parametros) [: tipo-de-retorno] [{propriedades}]

Boa parte é opcional!

Entretanto, quanto mais completo, melhor...

Cuidado 1

Métodos sempre escritos com iniciais em minúsculo.

```
getPatrocinadores() : Patrocinador[]
```

Cuidado 2

Se for retorno booleano, escrevam o nome do método de forma que essa semântica fique clara.

```
temPendencia(cpf : CPF) : Boolean
```

Cuidado 3

Há restrições? Particularidades, nessa parte comportamental?
+ setIdade(idade : Integer) {idade >= 18}

Diagrama de Classes - Relacionamentos

No nosso esboço inicial, usamos muitas associações.

Entretanto, é comum termos relacionamentos de diferentes tipos.

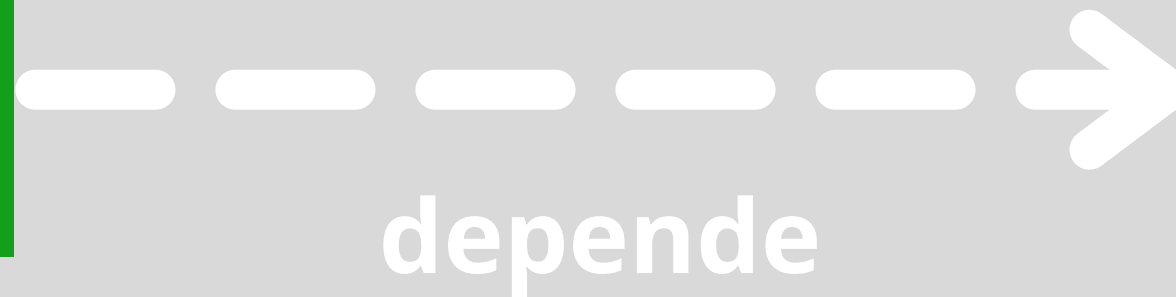
- Dependência
- Associação
- Agregação
- Composição
- Generalização
- Realização

Diagrama de Classes - Dependência

Dependência

- Não precisa especificar semântica, bastando usar corretamente a seta indicada na notação UML.
- Relacionamento comumente encontrado nos projetos Orientados a Objetos.
- Podem ser, portanto, muitas vezes, omitidos, no intuito de evitar falta de legibilidade no modelo.

Pesquisa



depende

FiltrosDePesquisa

Associação

Diagrama de Classes - Associação

- É o segundo tipo de relacionamento mais comumente encontrado.
- Já exploramos bem esse relacionamento no nosso exemplo de PetShop.
- Entretanto, reparem na cardinalidade desse modelo.
- Um cliente pode abrir uma ou até 3 contas. Tem um limitante nesse processo de abertura de conta. Portanto, é algo a ser respeitado na implementação

- Podemos pensar em um array, limitado em 3, em termos de multiplicidade, e implementado no Cliente, para tratar as Contas.
- Já na Conta, deve constar um atributo simples, visando tratar o Cliente associado.
- Reparem ainda que esse modelo não permite uma mesma conta ser associada a mais de um cliente. Sendo assim, não permite conta com múltiplas titularidades, ou contas conjuntas.

- Por fim, da forma como está especificada a navegabilidade do modelo, de Cliente para Conta, só será implementado esse sentido de navegação. Sendo assim, a partir de um Cliente, pode-se recuperar a(s) Conta(s) associadas.
- Se quiser, a partir de Conta, obter o Cliente associado, precisa usar:



Agregação

Diagrama de Classes - Agregação

- Relacionamento TODO-PARTE
- PARTE existe SEM o TODO
- Precisa avaliar cardinalidade.
- Em termos de implementação, em Java por exemplo, assemelha-se muito à implementação de uma associação.
- Entretanto, a semântica é muito diferente!
- "Losango" vazio...

- No caso, como a cardinalidade é 1 para *, na implementação:
 - No Departamento (TODO), precisamos manter um array ou uma lista, para tratar os vários Professores que podem fazer PARTE desse Departamento.
 - No Professor (PARTE), basta manter um atributo simples para tratar o Departamento (TODO), ao qual um dado Professor faz PARTE.
 - Aqui, pensando em navegabilidade em ambos os sentidos.

Departamento

1

tem

*

Professor

Composição

Diagrama de Classes - Composição

- Relacionamento TODO-PARTE
- PARTE NÃO existe SEM o TODO
- Precisa avaliar cardinalidade.
- Em termos de implementação, em Java por exemplo, difere-se bem em relação à Associação e à Agregação.
- Relacionamento Acoplamento MAIS ALTO.
- "Losango" cheio...

com

- No caso, como a cardinalidade é 1 para *, na implementação:
 - No Livro (TODO), precisamos manter um array ou uma lista, para tratar os vários Capítulos de Livro que podem fazer PARTE desse Livro.
 - No Capítulo de Livro (PARTE), basta manter um atributo simples para tratar o Livro (TODO), ao qual um dado Capítulo de Livro faz PARTE.
 - Aqui, pensando em navegabilidade em ambos os sentidos.

- A implementação não termina por aí. De preferência, no Construtor do Livro, se estivermos em Java por exemplo, devemos manter o controle sobre a(s) instância(s) de Capítulo de Livro.
- Portanto, para se ter acesso ao Capítulo de Livro, primeiramente, um Livro precisa existir. Quando uma instância de Livro é criada (TODO), então, uma ou mais instâncias de Capítulo de Livro podem ser criadas (PARTES), passando como parâmetro a instância do Livro (TODO - FORTE ACOPLAMENTO).

Livro

1

é_composto_por

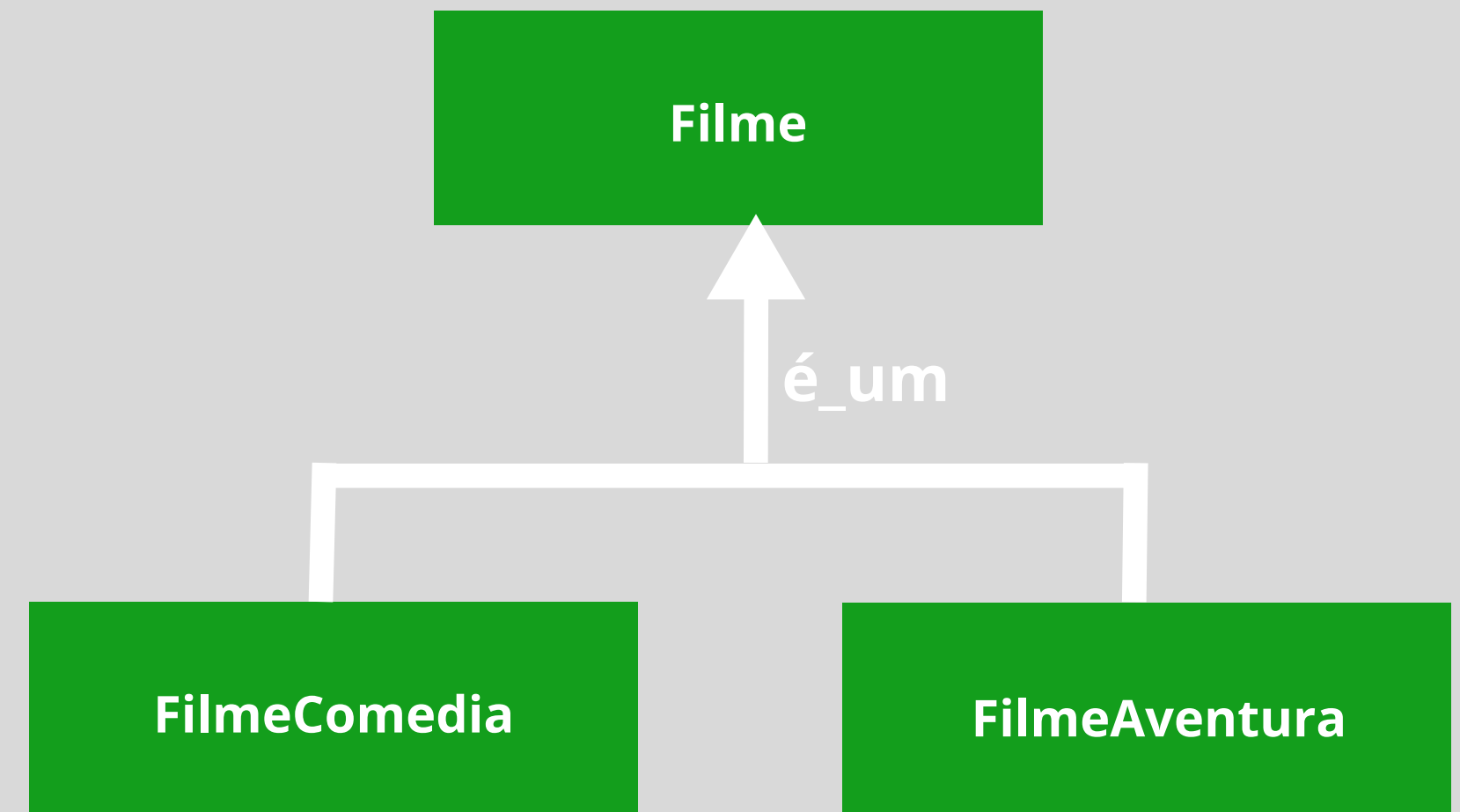
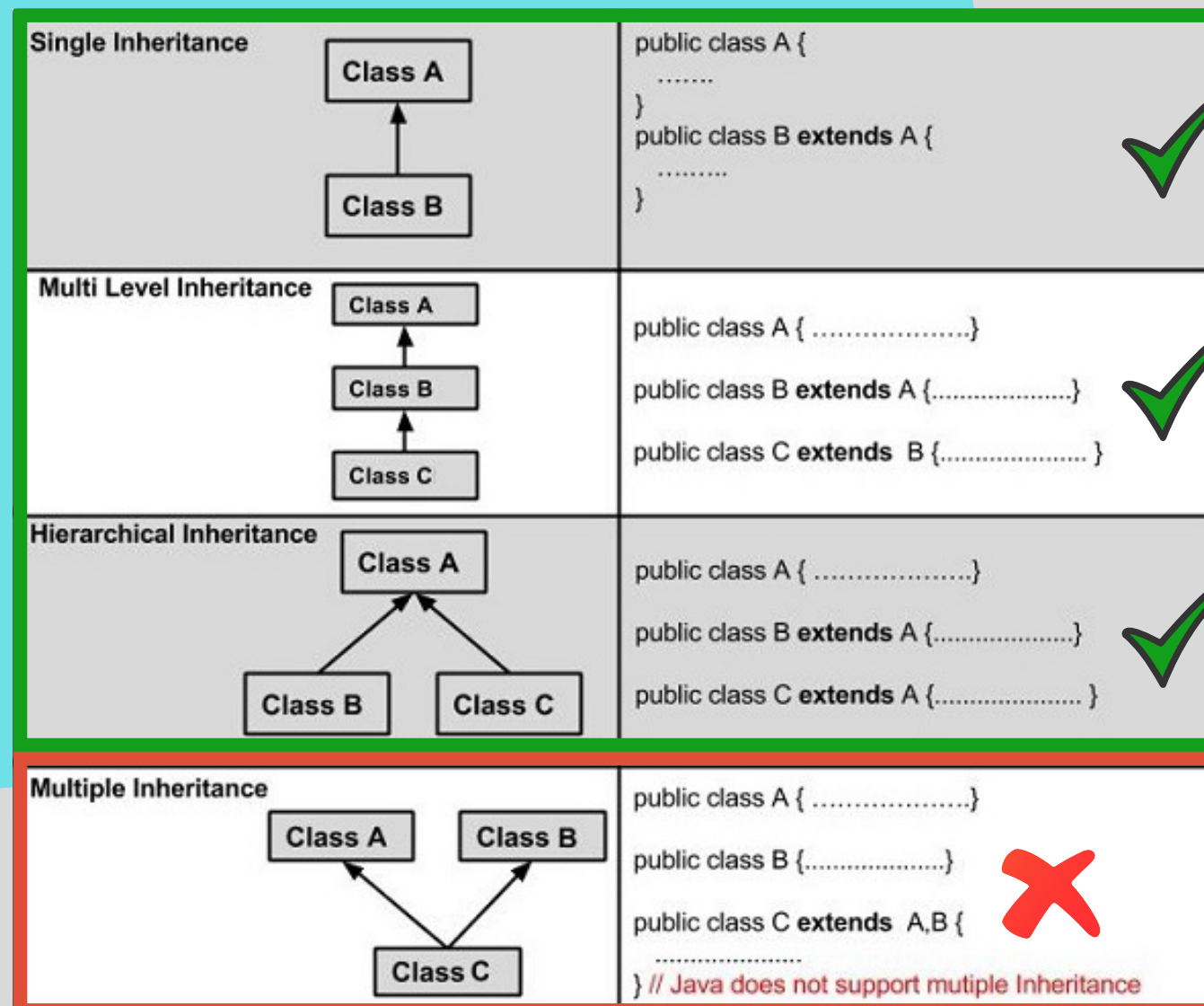
*

CapituloDeLivro

Generalização

Diagrama de Classes - Generalização

- Relacionamento Herança.
- Semântica é_um.
- Classe Pai ou Super, mais genérica.
- Classes Filhas ou Subclasses, mais específicas.
- "Linha" cheia...
- Implementando...
 - Em Java, tem-se o conceito de *extends*.



• Estudem sobre Classes Abstratas e Classes Concretas

• Estudem sobre Polimorfismo

• Generalização é um relacionamento que pode impactar em desempenho!
• Leiam sobre...

Realização

- Relacionamento Realização.
- "Linha" pontilhada...
- Implementando...
 - Em Java, tem-se o conceito de *implements*.

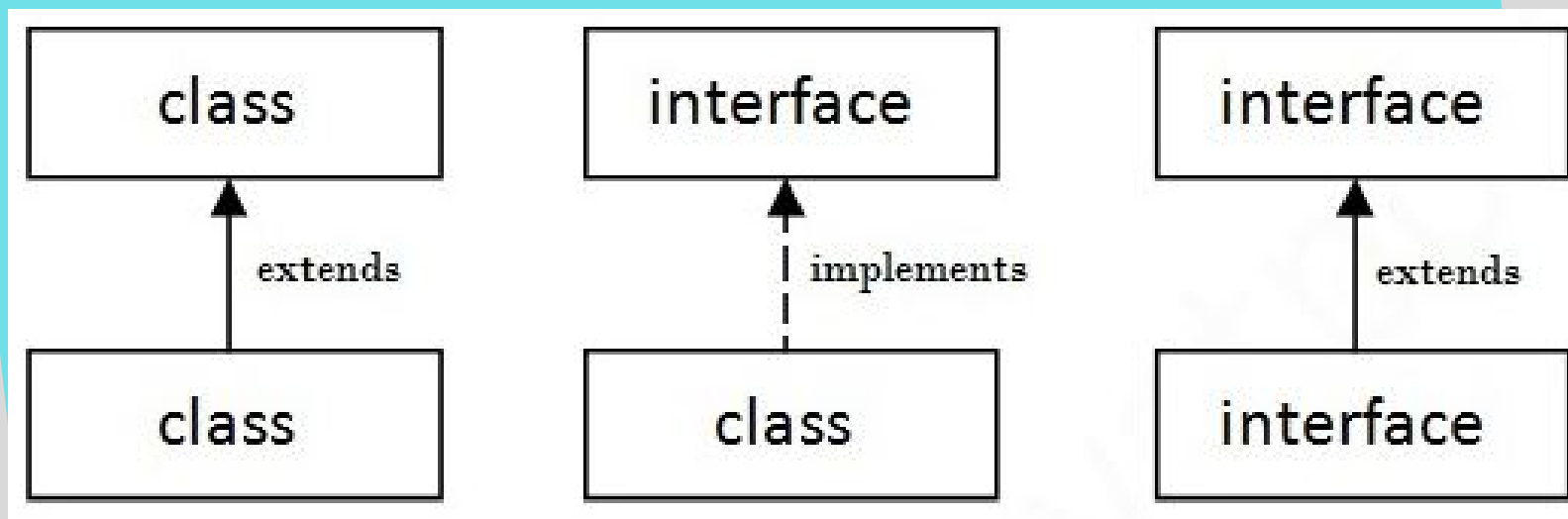
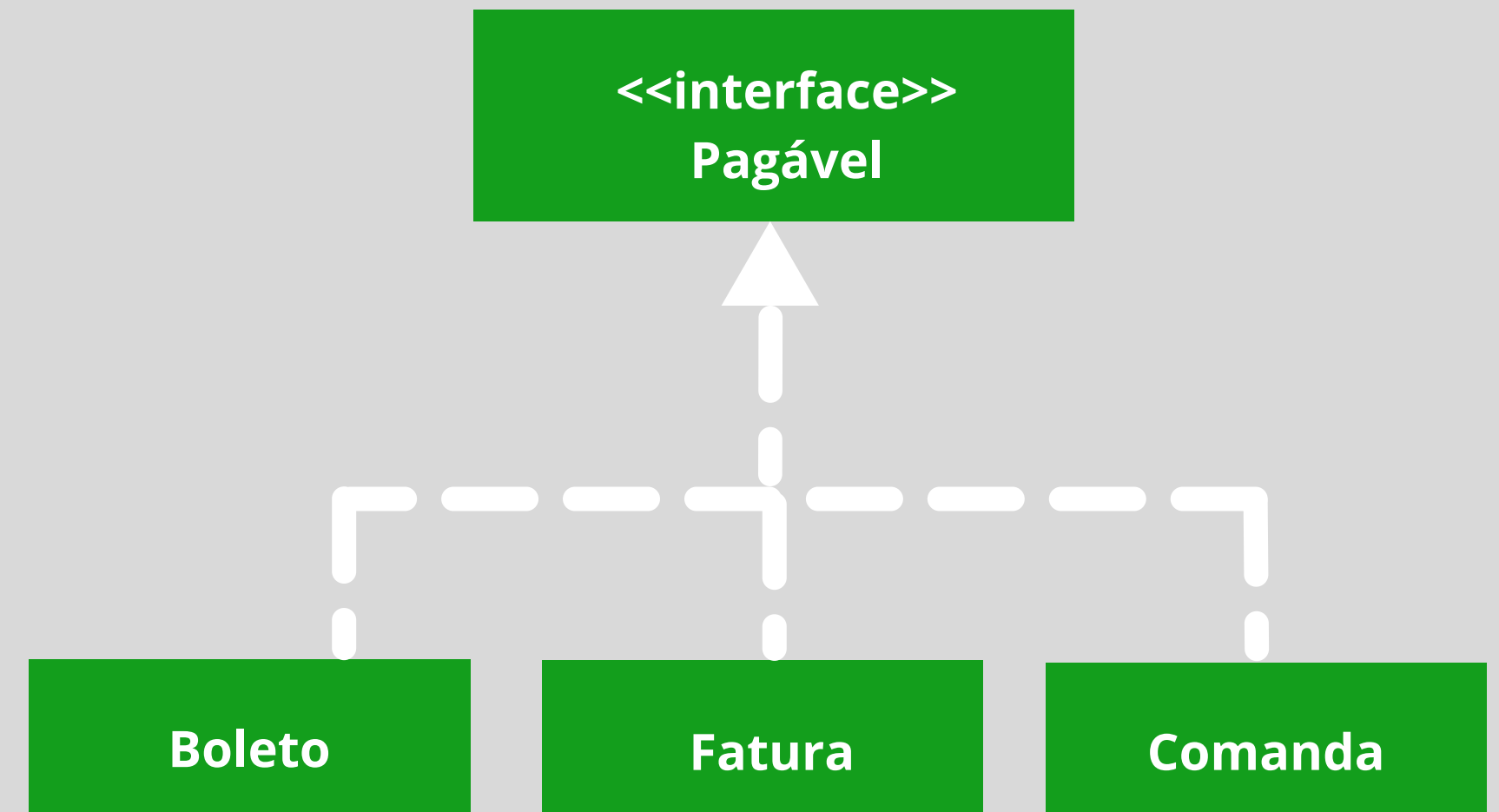


Diagrama de Classes - Realização



- Estudem sobre interfaces padronizadas

- Estudem sobre APIs

- Plugins, Componentes, Bibliotecas, e outros recursos reutilizáveis de programação usam o conceito de interface.
- Leiam sobre...

Cuidado 1

Se a cardinalidade dos relacionamentos de Associação/Agregação/Composição muda, a modelagem muda, e a implementação muda também. Portanto, avaliem as cardinalidades com cuidado!

Cuidado 2

Há muito mais sobre relacionamentos, na notação UML. Aqui, é apenas para conferir uma Visão Geral. Acessem as vídeo-aulas do Módulo DSW (Modelagem) para saber mais! Caprichem nos estudos! :)

Cuidado 3

Se o Diagrama de Classes for ficando muito extenso, modularizem a modelagem, e elaborem mais de um Diagrama de Classes. Depois, no Diagrama de Pacotes, ou no Diagrama de Componentes, enfatizem as correlações entre esses módulos/componentes.

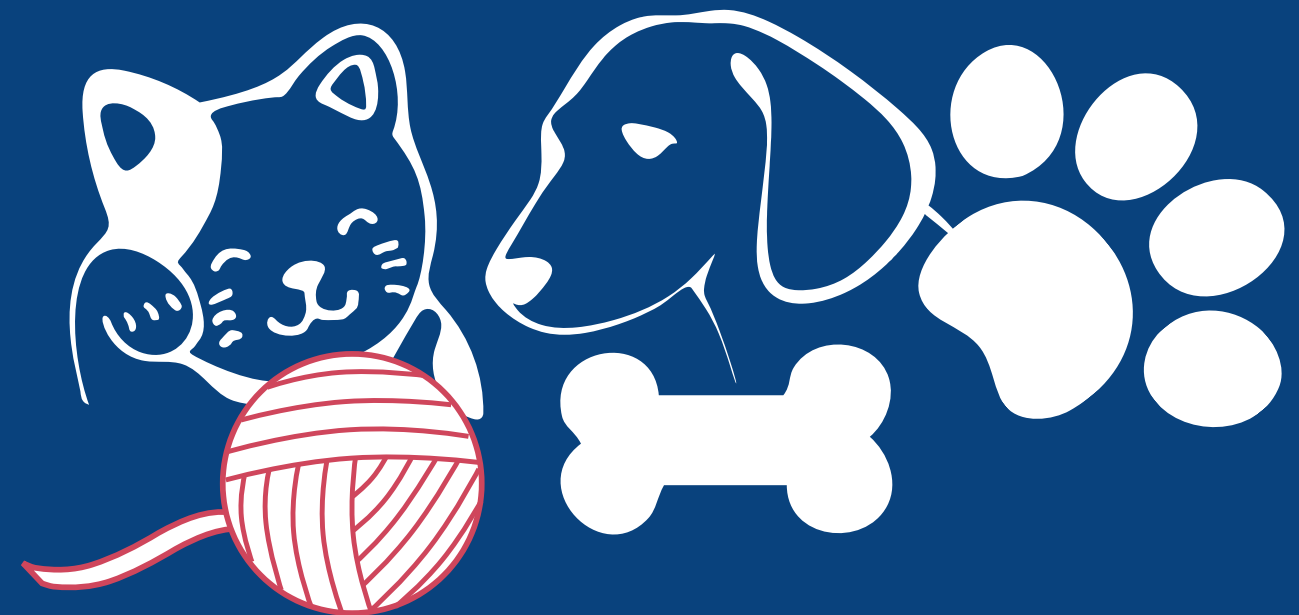
Diagrama de Pacotes

Para tornar o conhecimento sobre esse nível de modelagem mais concreto, vamos pensar no mesmo software que usamos anteriormente, no caso: PetShop!

Se esse PetShop será um site, podemos pensar em implementá-lo como uma típica aplicação Web. Além disso, um Estilo Arquitetural inerente em uma aplicação Web é o N-Camadas. Para mantermos tudo bem padronizado, vamos usar o Padrão Arquitetural MVC.

Nesse sentido, podemos modelar, usando o Diagrama de Pacotes, um esboço inicial das principais camadas do projeto.

Vamos lá...

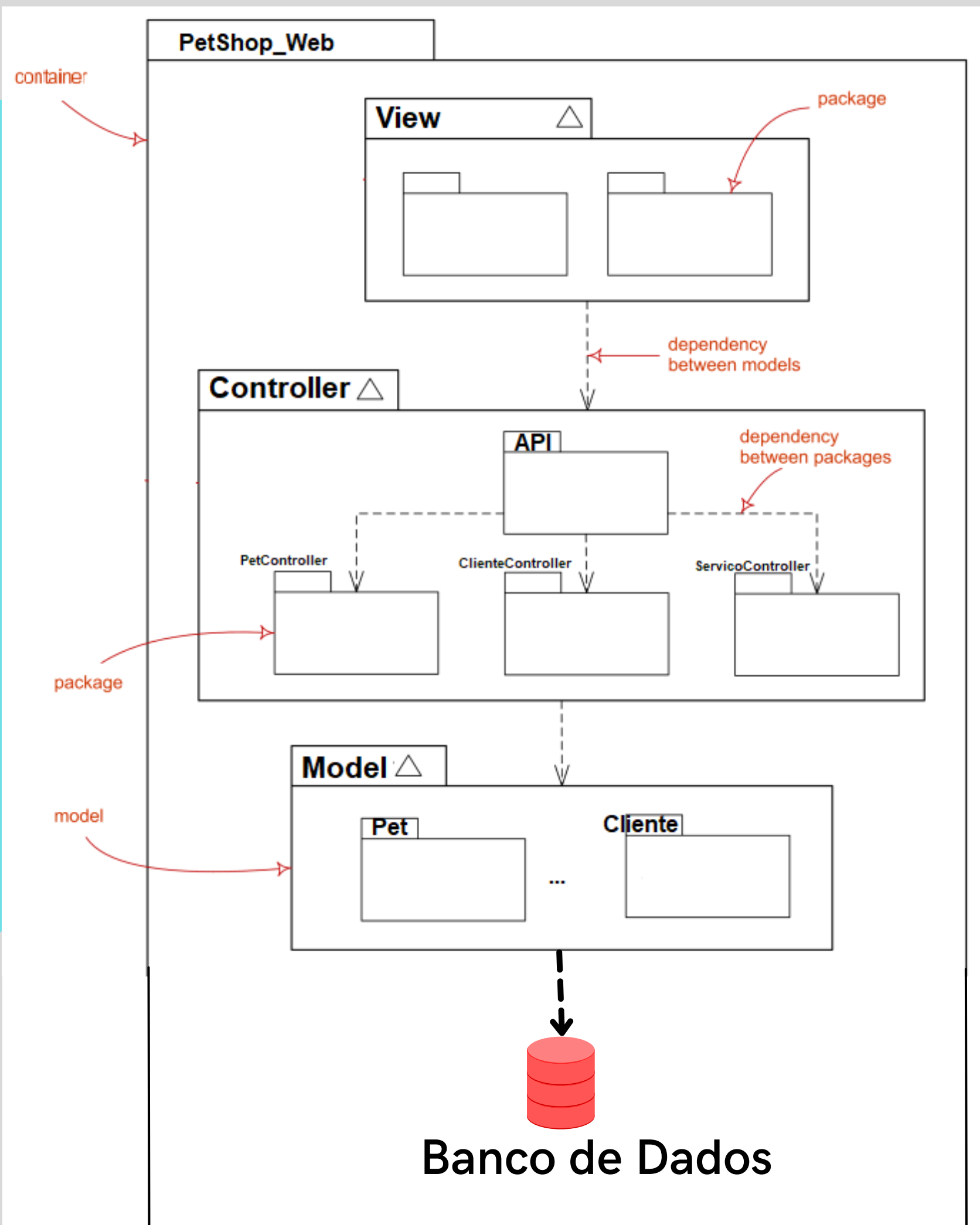


- Visão Estática
- Organizacional
- "Espelho" da Organização do Código
- Modelo ainda é um esboço, mas já contempla detalhes interessantes...

Por exemplo, podem ser acrescentadas semânticas, tais como:

- <<use>>
- <<merge>>
- <<access>>
- <<import>>

Diagrama de Pacotes



Cuidado 1

O modelo anterior é apenas um esboço, até para a nossa aula ser um resumo... Mas, podem caprichar bem! Evidenciando as várias entidades da camada Model (as entidades do Diagrama de Classes são um bom começo); as várias controladoras da camada Controller (múltiplas controladoras evitam sobrecarga de "controller fachada"), bem como os participantes da camada View (css, html, ...).

Cuidado 2

Especial atenção à especificação das relações de dependência entre as camadas! Estudem os conceitos de Arquitetura N-Camadas Relaxada & Arquitetura N-Camadas Restrita.

Cuidado 3

Lembrando que esse Diagrama de Pacotes é "espelho" da estrutura de pastas do código, em concordância com o Guia de Estilo. Acessem as vídeo-aulas do Módulo DSW (Modelagem) para saber mais! Caprichem nos estudos! :)

Diagrama de Componentes

Para tornar o conhecimento sobre esse nível de modelagem mais concreto, vamos continuar com o PetShop!

Modularizando em componentes...

Vamos lá...

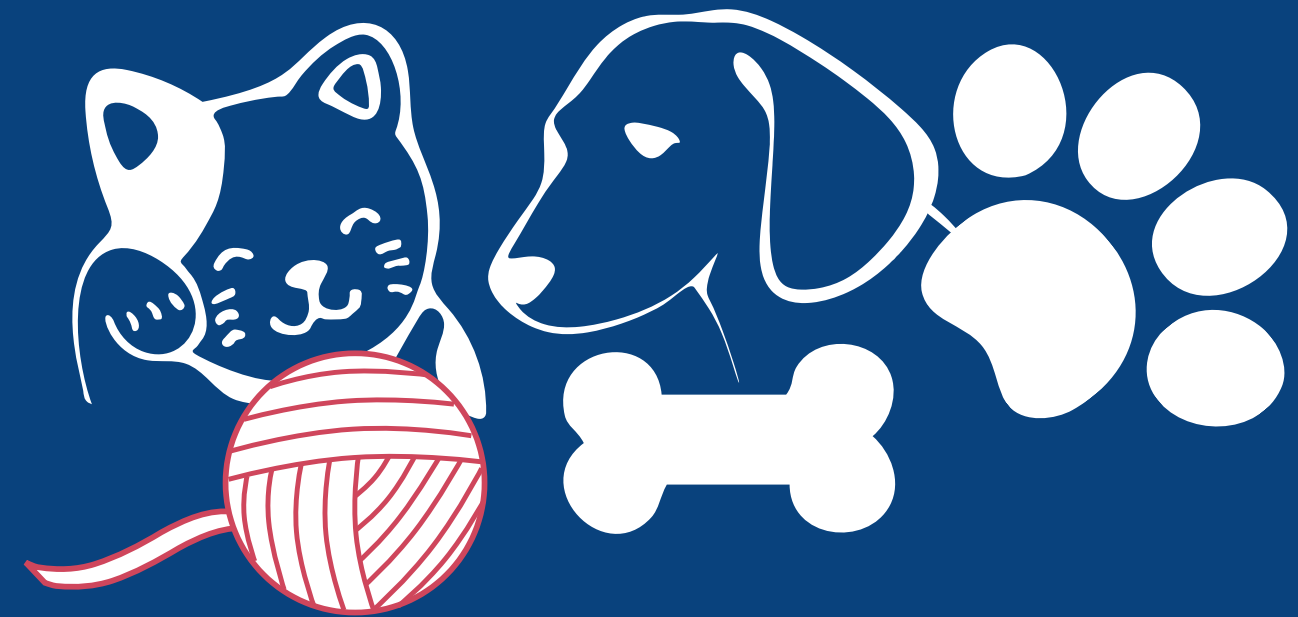
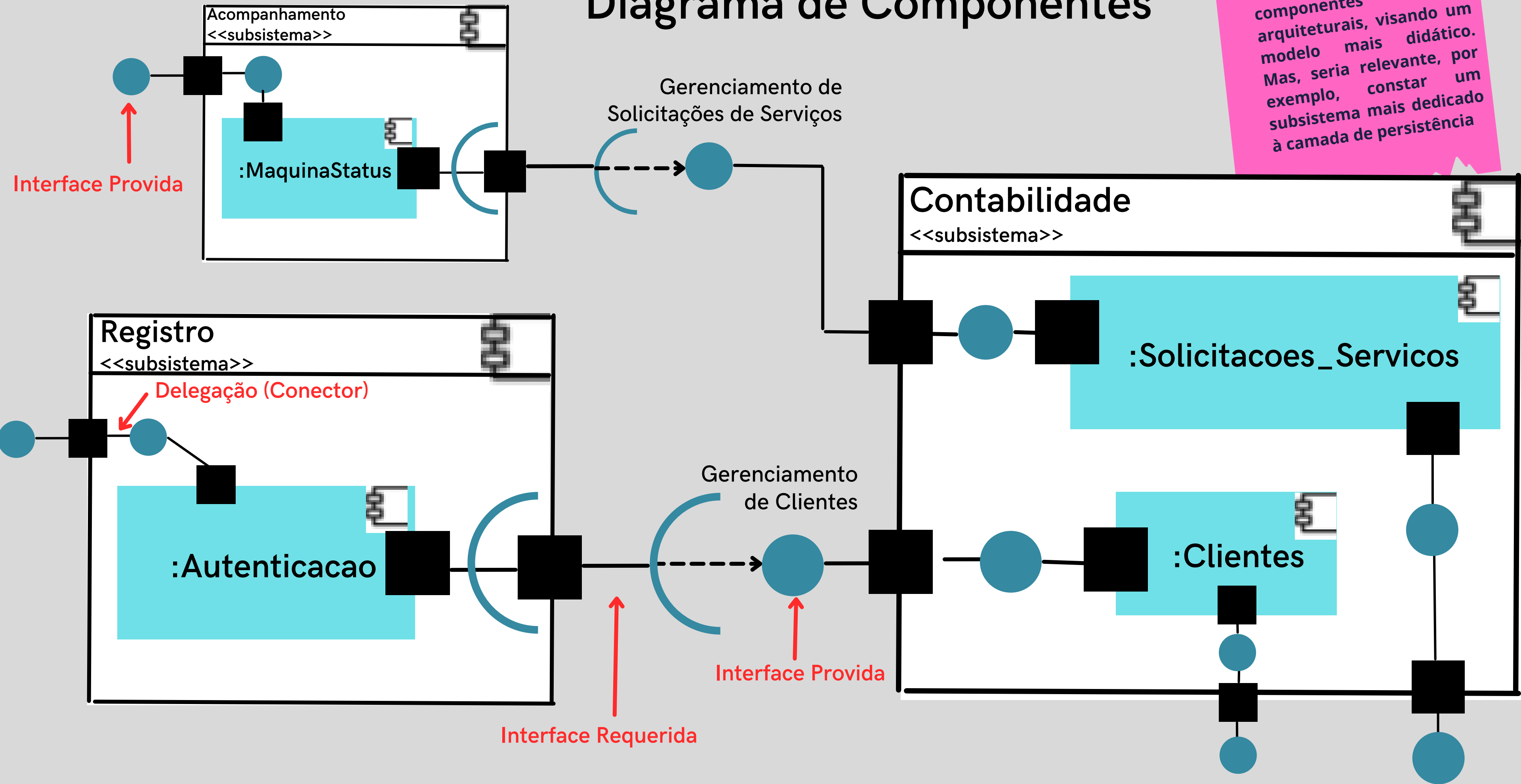


Diagrama de Componentes

• Há omissão de alguns componentes arquiteturais, visando um modelo mais didático. Mas, seria relevante, por exemplo, constar um subsistema mais dedicado à camada de persistência

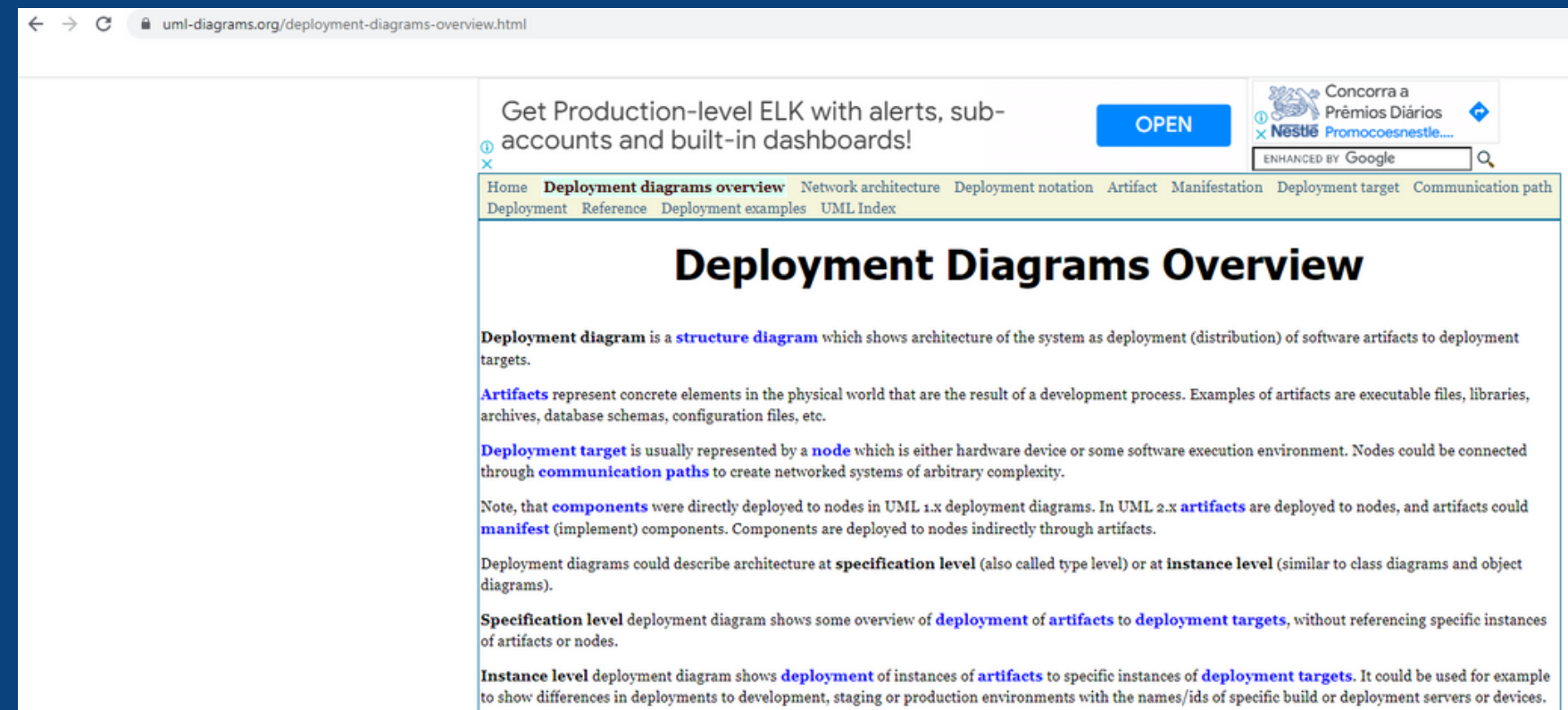


Dica!

Há ainda o Diagrama de Implantação, que também é um diagrama estático de suma relevância na documentação de um sistema, uma vez que revela os nós de implantação e seus protocolos.

Consultem, por gentileza:

<https://www.uml-diagrams.org/deployment-diagrams-overview.html>



Módulo Projeto Orientado a Abordagens Tradicionais

Conteúdos

Diagramas Dinâmicos

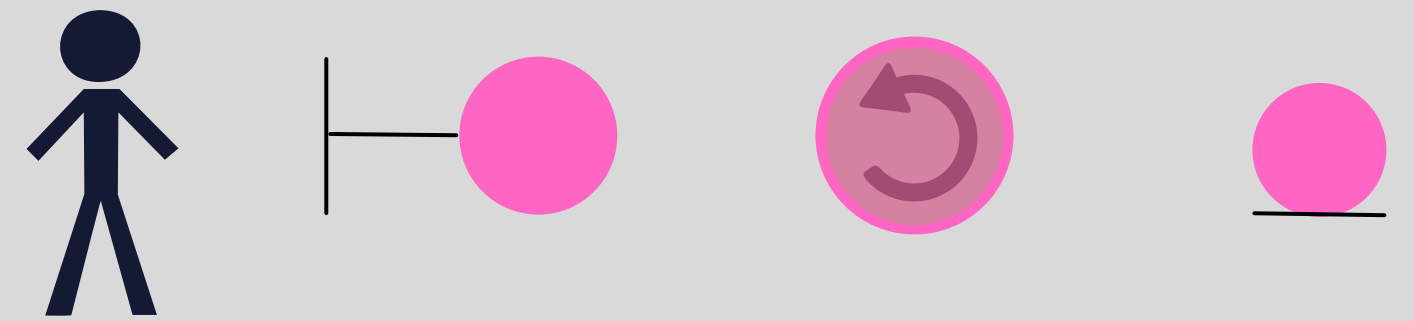
Artefatos Relevantes:

- Diagrama de Sequência
- Diagrama de Comunicação
- Diagrama de Atividades
- Diagrama de Estados

Há outros diagramas dinâmicos relevantes.
Entretanto, ficaremos nesses principalmente...



Diagrama de Sequência



- Diagrama de Cunho Dinâmico.
- Revela detalhamento temporal.
- Normalmente, focados nos fluxos principais, e nos fluxos de alternativos, e nos fluxos de exceção de um sistema.
- Pode ainda representar, por exemplo, as camadas de um sistema que usa uma arquitetura em N-Camadas, tal como o PetShop.

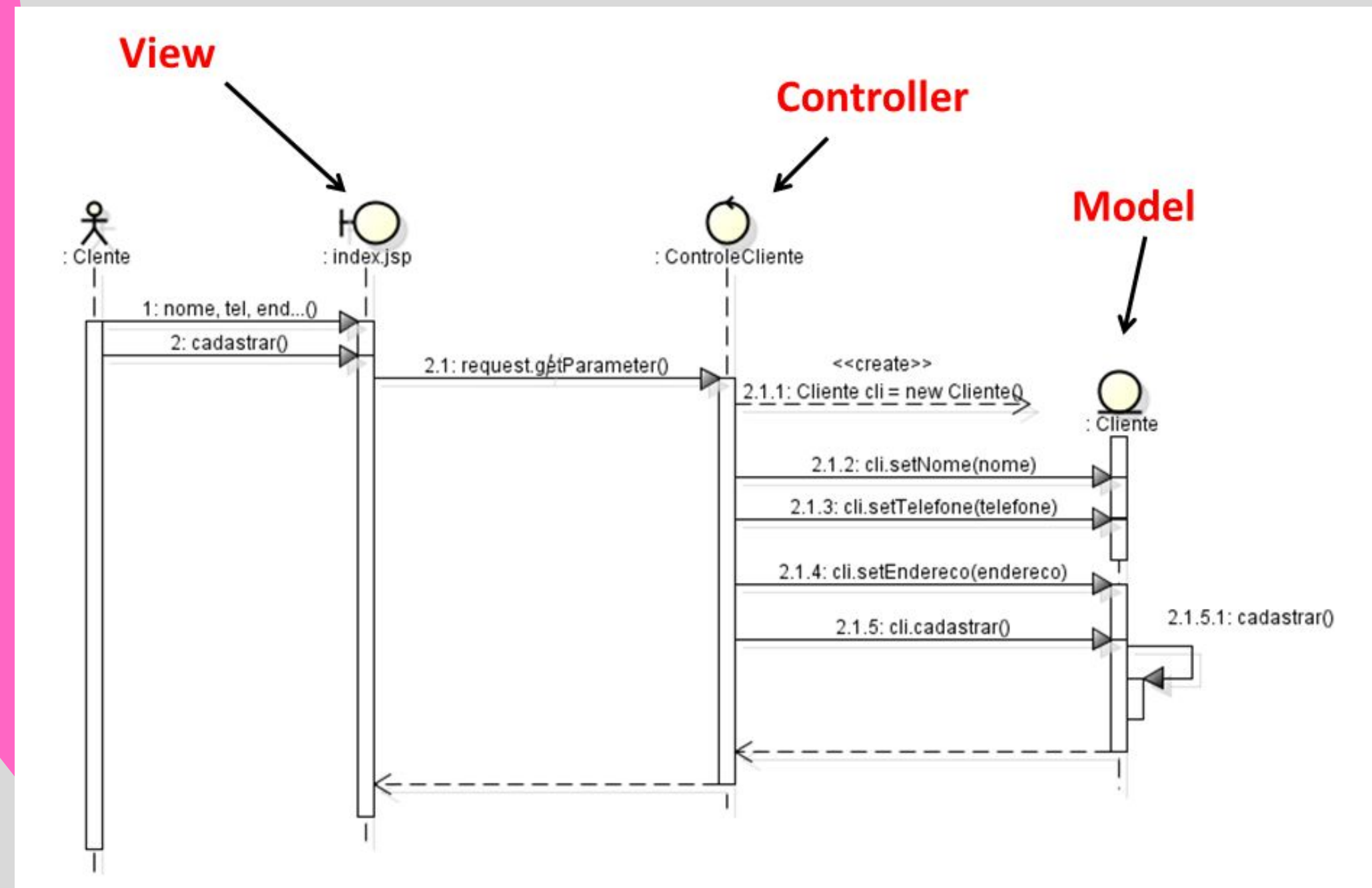


Diagrama de Comunicação

- Diagrama de Cunho Dinâmico.
- Revela detalhamento de comunicação, até mesmo no nível das instâncias.
- Deve-se marcar a ordem das ações que estão sendo modeladas na diagramação.

SELECT status FROM TabelaServico [WHERE condição];

1*:solicita_status()



:PetShop_Web

1.1:get_status()



:Servico

1.2:consulta_status()



:API_Banco

1.3:query_consulta



Banco de Dados

Diagrama de Atividades

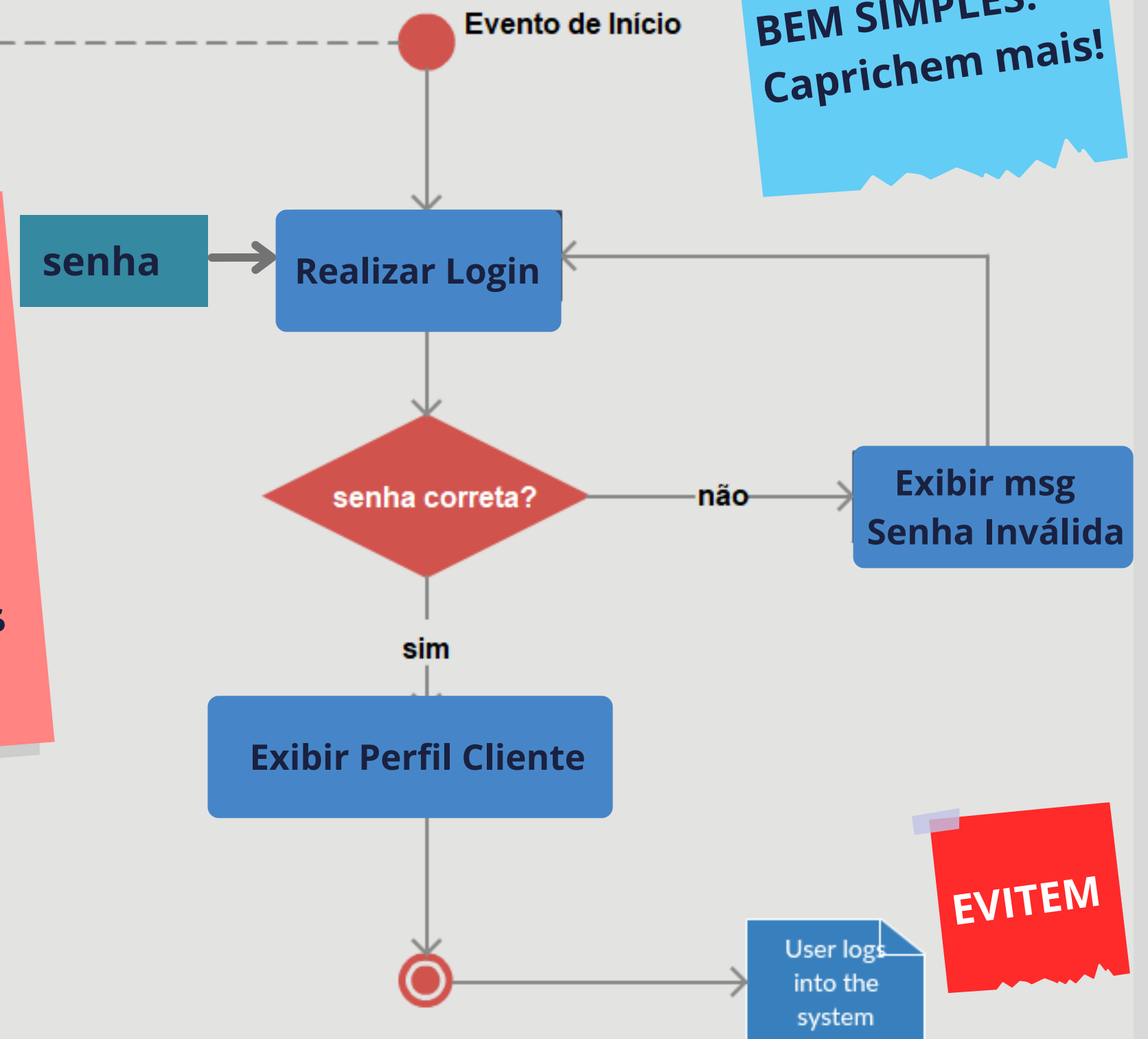
- Diagrama de Cunho Dinâmico.
- Como vocês modelaram o processo metodológico, em BPMN, têm-se muitas semelhanças entre as abstrações utilizadas lá e as utilizadas aqui, no Diagrama de Atividades, com a Notação UML.
- Foco na modelagem de um processo com atividades, eventos, subprocessos, e outras abstrações.

Atividades, retângulos com os cantos arredondados.

Recursos, insumos, como a senha, retângulos com os cantos retos.

Podem ocorrer variações, dependendo da ferramenta

Modelo BEM SIMPLES! Caprichem mais!



EVITEM

Dica!

Caprichar no uso de:

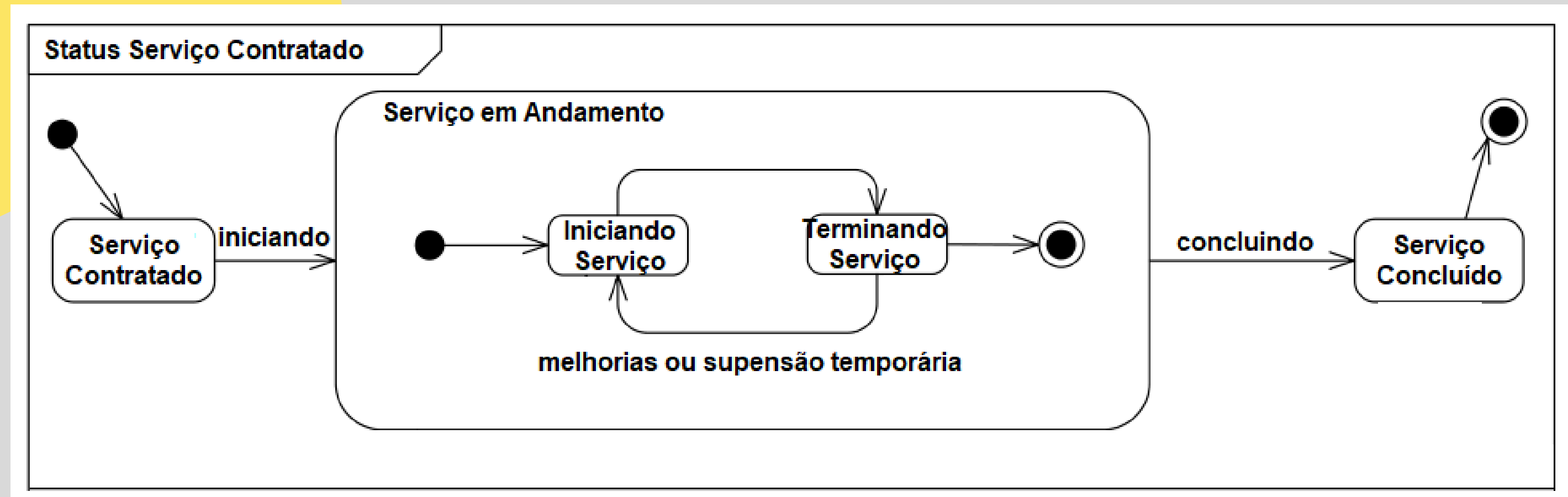
- eventos intermediários;
- partições;
- bifurcações;
- uniões, e
- Outras abstrações.



Diagrama de Estados

- Diagrama de Cunho Dinâmico.
- Revela detalhamento de uma típica máquina de estados.
- O caso da mudança de status, quanto ao andamento da solicitação do Cliente, na aplicação Web PetShop, justifica o uso de um Diagrama de Estados.

Foco em mostrar ao cliente sobre o status do **Serviço Contratado**, evidenciando seu **INÍCIO** e **TÉRMINO** (enquanto em **ANDAMENTO**).
Por fim, comunicando de fato sobre o **Serviço Concluído**.



Caprichem! :)

